秘密分散を用いたLUTベースノイズサンプリング

関口 貴生[1,a)]　千田 浩司[1]

概要：差分プライバシーのためのノイズは，マルチパーティ計算においては誰もが単独では知り得ないセキュアノイズサンプリングによって生成されなければいけない．サンプリング手法は様々研究されており，対象となるノイズの確率分布の種類によっても異なる場合がある．その中で Look-Up Table(LUT) を用いた方法は，離散化した確率分布をテーブル数で表現することから任意の分布に利用できるという利点がある一方で，誤差の少ない分布にするためにはデーブルサイズが大きくなるという問題があったが，近年の研究で LUT を複数層構成して互いに関連付けることでテーブルサイズを大幅に削減できることが示されている．本稿では複数層の LUT からセキュアノイズサンプリングする手法として 2 者間の秘密分散を用いる方法を提案する．既存の完全準同型暗号 (FHE) を用いた手法では暗号文のバッチサイズ以下の任意個のノイズを生成する場合，その個数に依らず一定の処理時間がかかるのに対し，提案手法では Oblivious Transfer(OT) と秘密分散プリミティブの計算コストがノイズの個数に比例するため，有用性が高い．また，オフラインで事前処理を済ませることで，オンライン処理を軽量化でき，且つ LUT の層数に依らず 2 者間の通信が定数ラウンドになるといった優位性ついて言及する．

キーワード：差分プライバシー，セキュアノイズサンプリング，LUT，秘密分散

LUT based Noise Sampling via Secret Sharing

Takao Sekiguchi[1,a)]　Koji Chida[1]

Abstract: For differential privacy, noise must be generated in multi-party computation such that no single party in the system can determine it independently—a fundamental requirement for security. Various sampling methods have been studied to generate noise with different probability distributions. Among these, the Look-Up Table (LUT) approach offers the advantage of supporting arbitrary distributions through discretized probability distributions represented as lookup tables. However, this method faces a scalability challenge: table size increases significantly as approximation error decreases. Recent research has demonstrated that by constructing LUTs in multiple layers with hierarchical interconnections, table size can be substantially reduced.

In this paper, we propose a method for secure noise sampling from multi-layer LUTs using two-party secret sharing. Existing approaches using fully homomorphic encryption (FHE) require fixed processing time regardless of the number of noise samples generated from ciphertexts smaller than the batch size. In contrast, our proposed method exhibits computational complexity for Oblivious Transfer (OT) and secret sharing primitives that scales linearly with the number of noise samples, offering superior efficiency. Additionally, by precomputing time-consuming operations offline, we enable lightweight online processing with constant number of communication rounds regardless of the number of LUT layers.

Keywords: Differential Privacy, Secure Noise Sampling, LUT, Secret Sharing

1. 研究背景

世帯・個人統計やスマートフォンなどのデバイス統計，

[1] 群馬大学情報学研究科
Graduate School of Informatics, Gunma University
[a)] takao.sekiguchi@gunma-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

あるいは Federated Learning のような集約学習に代表されるように，個々のデータを収集・活用したサービスやアプリケーションの導入が加速している．これらのシステムでは複数の参加者 (マルチパーティ) が通信路を通して中間値や出力値を送受信することが考えられるため，個人情報保護の観点から，個人を特定されないための処理として差分プライバシーが有効である．

差分プライバシーを確保するためには，保護されるべき本来のデータの尤度を下げる必要があり，データに対して直接ノイズを加える方法や特定のデータを選択する確率を付与する方法が一般的である．いずれの場合においても，ある確率分布に従う集合から値を引くという処理が必要であるため，差分プライバシーを実現する上で目標となる確率分布をどう表現するかという方法論に行き着く．確率分布を表現するために用いられる方法として，整数個のテーブルの中に台に対応する確率個分値を格納した LUT や累積確率を離散化して値を格納した配列，あるいは逆関数などが挙げられる．この中で LUT を用いた方法は，任意の確率分布に適用でき，インデックスをランダムに選ぶだけで出力を得られるためサンプリングが容易だが，目標の確率分布に対する誤差を抑えるためにはテーブルサイズが膨大になるという課題があった．しかし近年の研究で LUT を複数層に分割し互いに関連付けることでそのサイズを削減できることが示されている．ここで，マルチパーティでデータ処理を行う場合には，これらの方法でサンプリングするノイズもデータと同様に秘匿されなければデータを逆算されるリスクが生じる．系の誰もが単独で知り得ないノイズを生成することをセキュアノイズサンプリングと呼ぶ．

1.1 関連研究

Olive Franzese et al. によって LUT を複数層に分割し木構造のように各層 (枝) を幹で結びつけることにより 1 次元の LUT で確率分布を表現する場合に比べ 2 次元の LUT では大幅にサイズを削減減出来ることが示されている [1]．筆者らは 2 次元の LUT からセキュアノイズサンプリングをする方法として，FHE で暗号化したワンホットベクトルと内積をとる手法を提案しているが，FHE の演算はそれ自体重く，演算深度によって法を決定しノイズを除去するブートストラッピングが必要となりさらに重い処理が加わる．FHE の演算深度は 2 次元 LUT のテーブル長 (列数) によって決まり，2 次元 LUT の層数 (行数) によってブートストラッピングの回数が決まるため，対象の確率分布によっては処理時間がかかる場合が考えられる．また，FHE 暗号文として採用されている BGV は整数多項式を用いて複数のノイズを一つの暗号文にパッキングすることでノイズ生成を並列化することが可能であるが，言い換えると一つのノイズを生成したい場合でも多項式次数個のノイズを生成するのと同じ処理時間がかかり，適用するアプリケーションによっては有用性が低くなることが考えられる．

1.2 本稿の成果

本稿ではより軽量でスケーラブルな手法として，OT と秘密分散プリミティブで構成された新たな 2 者間秘密分散プロトコルを提案し，既存手法との性能比較を行った．提案手法ではノイズのサンプル数に対して処理時間が比例するため，既存手法のような多項式次数で決まる処理時間のボトルネックは存在しない．サンプル数ごとの処理時間及びデータ通信量は既存手法より軽量で，特にセットアップやオフライン処理に事前処理をまとめることでオンライン処理を軽量化することができる．また，通信ラウンド数は LUT の層数に依存せず $\mathcal{O}(1)$ となりレイテンシを抑えることができる．

このように提案手法では差分プライバシー確保のためにノイズを加える必要のある種々のサービスやアプリケーションのニーズに合わせてより柔軟に設計することが可能となる．

2. 差分プライバシー

差分プライバシー (DP) は，個人のデータを含むデータセットに対してクエリを実行する際，個人のプライバシーを守りながら統計的な分析を可能にするための数学的フレームワークであり，データセットに特定の個人の情報が含まれるかどうかを出力から推測できないようにすることを目的としている．$M(\cdot)$ を DP メカニズムとしてデータが一つだけ異なる隣接データベース D_0, D_1 を入力したとき，以下の不等式を満たすと (ε, δ)-差分プライバシーが達成される [2]．

$$\Pr[M(D_0) \in S] \leq e^{\varepsilon} \Pr[M(D_1) \in S] + \delta \qquad (1)$$

このとき ε はデータ差分の程度であり，これが小さいほど差分がなくなりプライバシーは保護されるが一方でデータの有用性は低下する．逆に大きい場合は差分が大きくなり安全性が低下する代わりにデータの有用性は高まる．ε の性質として DP メカニズムを m 回繰り返した場合，その値は $m \cdot \varepsilon$ となり，しばしばプライバシーバジェットとしてシステム全体で許容されるプライバシー損失として定義される．δ は差分プライバシーが破れる確率であり，小さいほど安全性が高い．

隣接データベースのデータ差分を埋める方法として，ε を変数に持つ，ラプラス分布やガウス分布に従うノイズを統計データに加える方法 [2] や，あるいはカテゴリデータのような特定のクエリに該当するデータを抽出するような場合に，ε の重みをかけた確率でデータを選択する方法 [3] などが提案されている．

© 2026 Information Processing Society of Japan

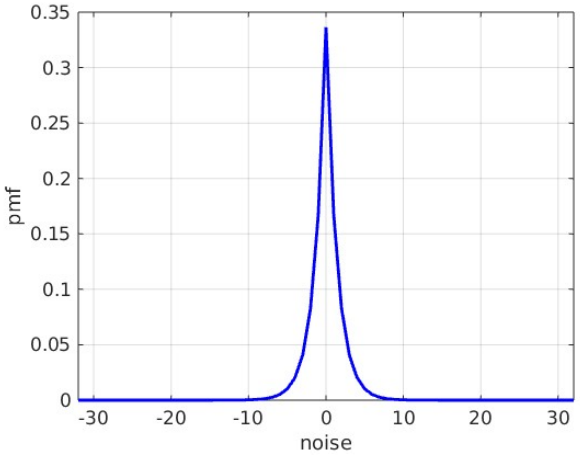


図 1 1d LUT によるラプラス分布
Fig. 1 Laplace dist. from 1d LUT

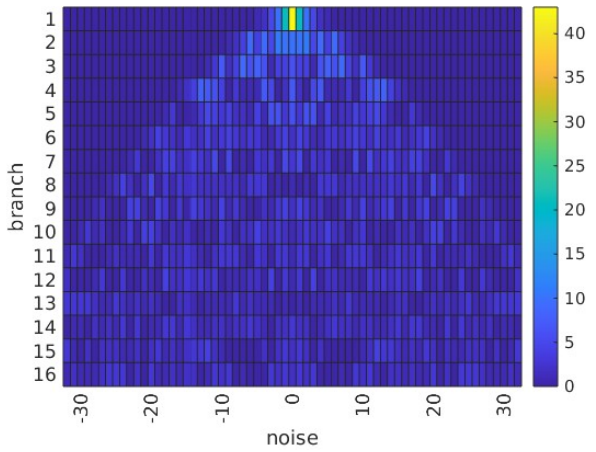


図 2 2d LUT のノイズカウント
Fig. 2 Noise counts on 2d LUT

3. LUT ベースノイズサンプリング

差分プライベートなデータを構成するためのノイズの確率分布は LUT によって表現することができる．ノイズ値の入ったテーブル数をテーブル総数で割った値がそのノイズ値の確率となるように LUT を構成するが，LUT のテーブル数は有限の整数なので，その中で目標の確率分布との誤差を，各システムで許容される式 (1) の δ 以内に収める必要がある．この LUT に対して一様ランダムなインデックスを選択してノイズサンプリングをすることができる．

3.1 1 次元 LUT

(ϵ, δ)-DP を満たす確率分布の 1 次元 LUT は解析的に生成することができる [4]．まずシステムの ϵ, δ 及び感度 Δ を決定し，各ノイズ値に対するテーブル数が要素となる配列 $\mathbf{A}$ を準備する．次に，$r \leftarrow e^{\epsilon/\Delta}$ を計算し，$r \geq 2$ ならば $A_0 \leftarrow 1$，それ以外は $A_0 \leftarrow \left\lceil \frac{1}{r-1} \right\rceil$ とし，$\mathbf{A}$ を A_0 で初期化する．このときの A_0 が確率分布の一番裾野にあるノイズ値のテーブル数となる．アルゴリズムのメインループではラプラス分布に代表される左右対称な分布を対象として，$\mathbf{A}$ の中央値に対して r をかけ，その値を新たに $\mathbf{A}$ の中央値とし，その左右のインデックスにそれぞれ $\mathbf{A}$ のインデックスの頭から中央値のインデックスまでの要素と中央値のインデックスから最後のインデックスまでの要素を挿入し $\mathbf{A}$ を更新する．ここで，δ は LUT のテーブルサイズ分の 1 より大きくなることから，メインループを繰り返してテーブルサイズがその条件を満たした場合，ループを抜けてアルゴリズムは終了する．出力された $\mathbf{A}$ の各要素を要素の合計値で規格化すると，(ϵ, δ)-DP を満たす確率質量関数が得られる．

図 1 に $\epsilon = 0.7, \delta = 1 \times 10^{-10}, \Delta = 1$ のラプラス分布をこの方法で 1 次元の LUT にした場合の確率分布を表す．このときテーブルサイズは約 1.5×10^{10} とかなり大きく，ノイズの取り得る値の範囲は $[-32, 32]$ の合計 65 通りとなる．

3.2 2 次元 LUT

3.1 で示したように，誤差 δ の小さい差分プライバシーを満たす LUT はそのテーブルサイズが膨大になる．これを 2 次元の木構造にすることでサイズを大幅に削減できることが示されている [1]．x をノイズ，目標の確率分布を $f(x)$ としてテーブルサイズ L の 1 次元 LUT で表される確率分布は $f_d(x) = \frac{freq_d(x)}{L}$ と表す．このとき，$freq_d(x)$ は要素 x の LUT 上の頻度であり，$\lfloor f(x)L \rfloor$ である．LUT は初めに全ての要素を $\perp$ で初期化し，次にこの頻度の数だけノイズ値 x と置き換えていくが，ノイズ x の値によっては頻度が 0 になるものもあり，またテーブルもすべてがノイズ値で埋まるとは限らない．値が入らなかったインデックスには $\perp$ が残る．この最初の LUT を根として，ここから根に入りきらなかった確率を同じサイズの LUT で構成し新たな枝とする．これを誤差 $f(x) - f_d(x)$ が十分小さくなるまで繰り返していくことで 2 次元 LUT を木構造で構成する．誤差を $Err[x]$ として $f(x)$ で初期化したとき，i 層での誤差は $Err[x] \leftarrow Err[x] - \frac{freq_{d_i}}{L} c$ で更新される．ここで c は i 層の確率分布の正規化係数であり，初期値を 1 とし $c \leftarrow c \frac{freq_{d_i}(\perp)}{L}$ で更新される．すなわち i 層の確率分布はひとつ前の $i-1$ 層で値が入らなかったテーブルの確率内に収めることができる．i 層では $Err_i[x] = \frac{Err[x]}{\sum_x Err[x]}$ で表現する誤差を初期化し，$freq_{d_i}(x) = \lfloor Err_i[x] \cdot L \rfloor$ としてノイズ x の頻度を計算する．

図 2 に 3.1 で生成したラプラス分布をテーブルサイズ 130，層 (枝) 数 16 の 2 次元 LUT で表現した場合のノイズカウントのヒートマップを示す．図のセルで表しているのは層ごとの各ノイズ値の頻度であり，数が多いほど青から

© 2026 Information Processing Society of Japan

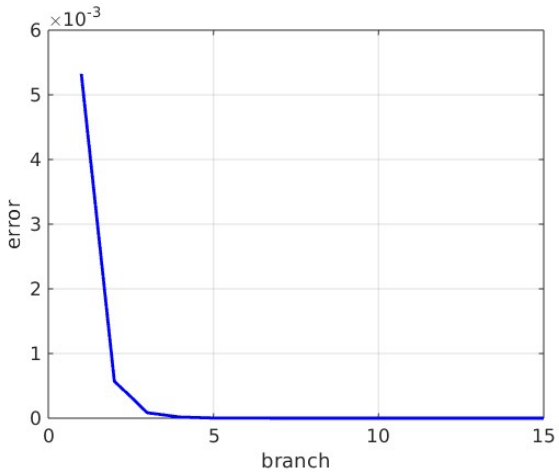


図 3 2d LUT の誤差
Fig. 3 Errors on 2d LUT

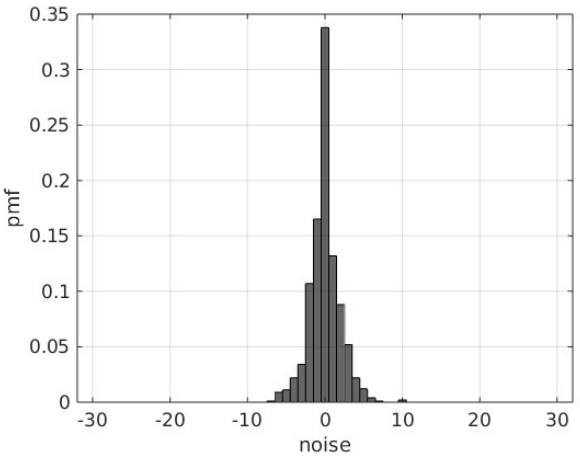


図 4 2d LUT からサンプルしたノイズ分布
Fig. 4 Noise dist. sampled from 2d LUT

黄色に近づく．これを見ると，3.1 で生成したラプラス分布では，一番上位の LUT はノイズ値が 0 付近の値で埋め尽くされていて，枝を増やして層を重ねていくごとに LUT に入るノイズの値のバリエーションが増えていくことが分かる．

一方で，図 3 は 2 次元 LUT の層に対する総誤差 $\sum_x Err[x]$ の値を示している．総誤差は層を増やすごとに減少していくことが分かり，$i = 12$ 層で約 8.4×10^{-11} 程度まで収束する．すなわち，1 次元 LUT でテーブルサイズ 1.5×10^{10} だったものを 130×12 の 2 次元 LUT で表現することができる．

3.3 ノイズサンプリング

3.2 で構成した 2 次元 LUT からノイズサンプリングするには各層ごとに LUT とランダムなワンホットベクトルの内積をとり以下の式で足し合わせる．

$$y_i = \sum_{T_i[j] \neq \perp} c_i[j] T_i[j] + \Big(\sum_{T_i[j] = \perp} c_i[j] \Big) y_{i+1} \qquad (2)$$

式の第一項は i 層の LUT のうち値の入っている部分との内積，第二項は $\perp$ が入っている LUT に対する演算を表す．もしワンホットベクトルの要素 1 が第一項にある場合は i 層の LUT の要素が引かれ，逆に要素 1 が第二項にある場合は $i+1$ 層すなわち下の層で選ばれた LUT 要素が引き継がれることになる．この漸化式を 2 次元 LUT の最下層から最上位の根まで繰り返すことで目標の確率分布からノイズサンプリングすることと等価になる．

図 4 はこの方法で 3.2 で生成した 2 次元 LUT からノイズサンプリングした場合のノイズ出力のヒストグラムであり，図 1 とおおよそ同じ分布になっていることが分かる．既存手法 [1] ではこれをセキュアノイズサンプリングにするためにワンホットベクトルを BGV 暗号文にして演算している．BGV 暗号文は多項式に複数の値をパッキングすることができるため，ワンホットベクトルを多項式次数分生成して一度に多項式次数分のノイズを出力することができる．またマルチパーティ間で式 (2) のテーブルを分割してそれぞれ演算を行い，一人のパーティが各パーティから出力の暗号文を受け取り，足し合わせて全パーティへ送り返すという処理を層ごとに繰り返す．

4. 秘密分散

秘密分散（Secret Sharing）は、暗号プロトコルにおける基本的なプリミティブであり、機密情報を複数の参加者に分割し、一定数以上の参加者が協力した場合にのみ元の情報を復元できるようにする技術である．本稿で紹介する提案手法では続く 3 つの秘密分散プリミティブを使用している．

4.1 Oblivious Pseudorandom Function (OPRF)

OPRF はクライアント (受信者) が入力，サーバー (送信者) が秘密鍵を持つ 2 者間プロトコルである．疑似乱数関数 (PRF) を F，インスタンス数を m，クライアントの入力を $(r_1, \ldots, r_m)$ としたとき，以下の手順でクライアントは自分の入力値をサーバーに知らせず，疑似乱数値を得ることができる [5]．このとき，秘密鍵はクライアントに渡らない．

- 送信者はランダムなシードから PRF のための秘密鍵 $k^*, k_1, \ldots, k_m$ を生成する．
- 受信者は自分の入力とサーバーが生成した秘密鍵から成る疑似乱数値 $F((k^*, k_1), r_1), \ldots, F((k^*, k_m), r_m)$ を得る．

これらの手順は OT を用いて 2 者間で対話的に実行される．サーバーは同じ秘密鍵を用いて任意の入力に対する PRF を生成することができるので，それらを秘匿したい

© 2026 Information Processing Society of Japan

データにマスクしてクライアントへ送信することで，クライアント側では OPRF で得られた PRF を用いて，自分の入力値の PRF でマスクされたデータのみを復元することが可能となる．

4.2 Distributed Point Function (DPF)

関数秘密分散（FSS）は Elette Boyle et al. により導入された手法で，与えられた関数族 $\mathcal{F}$ から関数を加法的に秘密分散する方法を提供する．具体的には，m 者 FSS 方式は，関数 $f:\{0,1\}^n \to G$ を，鍵 $k_1,\ldots,k_m$ で記述される関数 $f_1,\ldots,f_m$ に分割し，$f=f_1+\cdots+f_m$ を満たし且つ任意個の鍵で構成される部分集合によって f を算出できないようにする．この方法は (Gen, Eval) のペアのアルゴリズムによって実現され，Gen は m 個の鍵を生成し，Eval では $f_i(x)=Eval(i,k_i,x)$ でパーティ i の関数が計算できる．分散点関数 (DPF) はその特殊な場合であり，$\mathcal{F}$ が点関数族である場合に相当する．すなわち点関数 $f_{\alpha,\beta}$ は入力 α では β が出力され，他のすべての入力では 0 が出力される [6][7].

$$f_{\alpha,\beta}(x)=\begin{cases}\beta & \text{if } x=\alpha\\ 0 & \text{otherwise}\end{cases} \tag{3}$$

DPF 鍵は OT を用いて 2 者間で対話的に生成することができる [8].

4.3 PV プロトコル

ABY2.0(Arithmetic Boolean Yao) の PV プロトコルは 1 ビットのブール値 p と環上の値 v の積 pv を 2 者間でセキュアに計算するための仕組みである [9]．オフラインフェーズでは，双方がランダムな Beaver トリプルに相当する相関乱数を生成するが，ビット p にはブールシェアを，値 v には算術シェアを使うため，異なるシェアリング方式をまたいだ乗算が必要になる．オンラインフェーズでは，各パーティが自分のシェアをマスクして相手に送り，受け取った値と合わせて pv の算術シェアを再構成する．通常の Beaver トリプルや従来の ABY では Boolean to Arithmetic(B2A) 等の型変換を経由する必要があり，通信コストが高かったが，ABY2.0 ではこれを 1 ラウンド・定数通信量で解決している．

$$\Delta_v=v+\delta_v \tag{4}$$

$$p=\Delta_p^a+(1-2\Delta_p^a)\delta_p^a \tag{5}$$

$$pv=(\Delta_p^a+(1-2\Delta_p^a)\delta_p^a)(\Delta_v-\delta_v) \tag{6}$$

v のシェア δ_v と p の算術シェア δ_p^a は互いに既知の Δ_v,Δ_p^a を用いて式 (4)，(5) のように表され，式 (6) によって pv が計算できる．Δ_v,Δ_p^a は互いのシェアにマスクをして送受信した値から構成する．オフラインフェーズで準備したランダム相関から pv の算術シェアを計算する方法は A.1 に記載する．

5. 提案手法

提案手法では，i を LUT の任意の層 (任意の行)，j を任意のインデックス (任意の列) として，LUT2 次元 LUT を要素が $\perp$ か否かを表すフラグビット $b_i[j]$ と要素 $T_i[j]$ を組み合わせた行列として考え，$\perp$ の要素は 0 に初期化する．

$$\mathbf{T}=\begin{bmatrix}(b_0[0],T_0[0]) & \cdots & (b_0[2^l-1],T_0[2^l-1])\\ (b_1[0],T_1[0]) & \cdots & (b_1[2^l-1],T_1[2^l-1])\\ \vdots & & \vdots\\ (b_{\lambda-1}[0],T_{\lambda-1}[0]) & \cdots & (b_{\lambda-1}[2^l-1],T_{\lambda-1}[2^l-1])\end{bmatrix} \tag{7}$$

式 (7) を用いたとき，2 次元 LUT からノイズサンプリングすることは，各層から 1 つずつランダムにインデックス j_i を選び，その出力から次の式を計算することと等価になる．

$$\begin{aligned}&\sum_{i=0}^{\lambda-1}T_i[j_i]\prod_{k=0}^{i-1}b_k[j_k]\\ &=T_0[j_0]+T_1[j_1]\cdot b_0[j_0]+\\ &\cdots+T_{\lambda-1}[j_{\lambda-1}]\cdot b_0[j_0]\cdots b_{\lambda-2}[j_{\lambda-2}]\\ &\quad\text{where }\prod_{k=0}^{-1}b_k[j_k]:=1\end{aligned} \tag{8}$$

式 (8) では $\perp$ フラグが 1 のとき要素は 0 なので，例えば $i=0$ 層で引いたインデックスの要素が本来 $\perp$ の場合 $b_0[j_0]=1,T_0[j_0]=0$ となり T_1 以降の項が残り，逆に $i=0$ 層でひいたインデックスに値が入っていれば $b_0[j_0]=0,T_0[j_0]\neq 0$ となり T_0 の項だけが残る．フラグビットは一つでも 0 になればそれ以降の層の要素は反映されず，すなわち 2 次元 LUT を上から選んでいく操作に他ならない．

5.1 プロトコル

次に 2 者間での秘密分散を考えて，式 (7) の $\perp$ フラグを $b_i^{(0)}[j],b_i^{(1)}[j]$ の xor シェア，テーブルの要素を $T_i[j]+\tau_i,-\tau_i$ の加法シェアとした行列 T' をパーティ 1 が準備し，そこからパーティ 0 がインデックスを引き，式 (8) を計算する．ここで簡単のため，式 (8) を次のように書き換える．

© 2026 Information Processing Society of Japan

$$\sum_{i=0}^{\lambda-1} T_i[j_i]P_i[j_i] \tag{9}$$

$$\text{where } P_i[j_i] = \prod_{k=0}^{i-1} b_k[j_k], P_0[j_0] = 1$$

Protocol: OPRF+DPF+PV

オフライン処理:

(1) パーティ 0 は OPRF(KKRT) でパーティ 1 が持つ 2 次元 LUT T' の各層から自分の選んだインデックスの要素を取得.

(2) 各パーティは $\perp$ フラグのシェア $b_i^{(0)}[j], b_i^{(1)}[j]$ に対する乱数マスクビット $r_i^{(0)}[j], r_i^{(1)}[j]$ を生成し，これらの xor を入力とする DPF 鍵 $k^{(0)}, k^{(1)}$ を生成.

(3) PV プロトコル用の相関 $d_{xor}^{(0)}, d_{xor}^{(1)}, d_{add}^{(0)}, d_{add}^{(1)}, m^{(0)}, m^{(1)}, q^{(0)}, q^{(1)}$ を生成

パーティ 0 のオンライン処理 (パーティ 1 も同様):

(1) $b_i^{(0)}[j_i] \oplus r_i^{(0)}[j_i]$ を計算し、パーティ 1 へ送信.

(2) パーティ 1 から受け取った $b_i^{(1)}[j_i] \oplus r_i^{(1)}[j_i]$ を用いて $c_i[j_i] = b_i[j_i] \oplus r_i[j_i]$ 及びその整数表現 c_{idx} を計算.

(3) $k^{(0)}$ と $x \oplus c_{idx}$ を入力として DPF 評価し，$P_i^{(0)}[j_i]$ を得る.

(4) $P_i^{(0)}[j_i] \oplus d_{xor}^{(0)}[j_i]$ および $T_i^{(0)}[j_i] - m^{(0)}[j_i]$ を計算しパーティ 1 へ送信

(5) パーティ 1 から受け取った $P_i^{(1)}[j_i] \oplus d_{xor}^{(1)}[j_i]$, $T_i^{(1)}[j_i] - m^{(1)}[j_i]$ を用いて $P_i[j_i] \oplus d_{xor}[j_i]$, $T_i[j_i] - m[j_i]$ を計算.

(6) PV を計算して以下を得る.

$$\sum_{i=0}^{\lambda} T_i^{(0)}[j_i]P_i^{(0)}[j_i]$$

図 5.1 に本プロトコルの手順を示す．オフラインのステップ 1)〜3) は OT をベースにインタラクティブに実行する．ステップ 1) では OPRF を用いて 1 out of N OT を LUT の層分行うため λ 個の PRF 鍵が必要となるが，OPRF 送信者であるパーティ 1 は LUT の各層のインデックスをそれぞれの PRF 鍵で作成した PRF 値をまとめてパーティ 0 へ送信することでラウンド数は片側だけの 0.5 回で済む．続くステップ 2) では DPF 用に，各パーティから λ 層分の $\perp$ フラグに対するランダムマスクビットを入力としてこれらの xor が入力となる DPF 鍵を生成する．ここで $r_{idx} = (r_{\lambda-1}^{(0)}[j_{\lambda-1}] \oplus r_{\lambda-1}^{(1)}[j_{\lambda-1}]) \ll (\lambda-1) \,|\, \cdots \,|\, (r_0^{(0)}[0] \oplus r_0^{(1)}[0]) \ll 0$ であり，ランダムマスクビットの xor の整数表現である．ステップ 3) ではテーブルの要素と $\perp$ フラグのプレフィックス積との積のための相関を，$\perp$ フラグのプレフィックス積の xor シェアと加法シェア用に 1 種類ずつ，テーブル要素の加法シェア用に 1 種類，さらにそれらの積の加法シェア用にもう 1 種類準備する.

オンライン処理ではステップ 1) でそれぞれのパーティが自分の持つ $\perp$ フラグのシェアに DPF の入力となったランダムマスクのシェアを xor した値を送受信して 1 ラウンド通信する．ステップ 2), 3) で受信したデータを基に c_{idx} を計算し，$x \oplus c_{idx}$ をインデックスとして DPF タグを出力する．このとき，$x = b$ のとき $c_{idx} = r_{idx}$ になるので x を λ ビットの整数で走査すると $\perp$ フラグ $b_i[j_i]$ の真値で構成されるプレフィックス積 $P_i[j_i]$ の xor シェアを導出することができる．最後に，テーブル要素とプレフィックス積との積を PV プロトコルで計算するために，ステップ 4) でそれらのシェアにマスクをしたものを送受信して 1 ラウンド通信し，ステップ 5), 6) で $T_i^{(0)}[j_i]P_i^{(0)}[j_i]$ をローカルで計算して式 (9) の加法シェアを導出できる.

表 1 性能比較 @ WSL(Processor: 12th Gen Intel(R) Core(TM) i5-1245U 1.60 GHz, RAM: 32GB)

Table 1 Performance Comparison

	既存手法	提案手法
処理時間	90 [s]	17 [s]
処理時間 (/サンプル)	2.8 [ms]	0.5 [ms]
通信量	1487 [MB]	906 [MB]
通信量 (/サンプル)	45 [KB]	27 [KB]
ラウンド数	$\mathcal{O}(\lambda)$	$\mathcal{O}(1)$

6. 実装と評価

図 5.1 のプロトコルを libOTe[10] のライブラリを使用して実装した．比較対象となる既存手法の実装は OpenFHE, Efficient Multi-Party computation toolkit(EMP), Scalable Mixed-Mode MPC をライブラリとした Secure Noise Sampling in MPC for DP[11] を使用した．これらの実装を用いてテーブルサイズ 128，層数 5 の 2 次元 LUT から 2^{15} 個のノイズを 2 パーティでセキュアサンプリングした場合の性能比較を行った.

表 1 の結果を比較すると，提案手法では処理時間，通信量を削減できることが分かる．本比較におけるサンプル数の 2^{15} とは BGV の多項式次数であり，すなわち既存手法においてはこのサンプル数以下のノイズを生成する場合においても同様の処理時間がかかる．ここで，多項式次数を下げる場合には安全性の要求を考慮する必要がある．また，式 (2) は漸化式となっていることからわかるように，既存手法は 2 次元 LUT の上下の隣り合う層の結果を次の入力とする必要があるため，全層の LUT を処理するのに必要な通信ラウンド数は $\mathcal{O}(\lambda)$ になるが，提案手法では層

© 2026 Information Processing Society of Japan

表 2　処理時間内訳
Table 2　Processing Time Breakdown

	処理時間
セットアップ	2.2 [s]
セットアップ (1 サンプル)	24 [ms]
オフライン	13.6 [s]
オフライン (1 サンプル)	0.49 [ms]
オンライン	0.9 [s]
オンライン (1 サンプル)	46 [μs]

数 λ に依存せず，5.1 で述べた定数ラウンドになる．

表 2 に処理時間の内訳を示す．セットアップには baseOT と OTe(IKNP) が含まれる．libOTe の baseOT は楕円曲線や CRYSTALS-Kyber(ML-KEM) で構成されるため，処理が重いが一度作れば IKNP 拡張でランダム相関をたくさん作ることができる．そのため単一ノイズをサンプリングするには baseOT にかかる時間が支配的になるが、サンプル数を増やすほどセットアップの処理時間は IKNP 拡張の恩恵を受け，サンプルあたりの処理時間を抑えることができる．オフライン処理，オンライン処理は図 5.1 のオフライン，オンラインに相当する処理を実行している．オンライン処理は式 (9) の計算に相当するが，オフラインにその事前準備を割り当てると処理時間全体の約 6%で実行することが可能である．

7. 結論

本稿では 2 次元の LUT からセキュアノイズサンプリングする手法として 2 者間の秘密分散を用いる方法を検討した．その結果，提案手法は既存手法である BGV を用いた方法に比べ，処理時間，通信量及び通信ラウンドにおいて優れており，且つサンプリング数や手順に応じた設計が可能な柔軟性があることを示した．今後は他に考え得るナイーブな手法との比較，種々の確率分布への適用，並列化や簡略化による提案手法の改良などを検討していきたいと考える．

謝辞　本研究は，JST 経済安全保障重要技術育成プログラム【JPMJKP24U5】の支援を受けたものです．

参考文献

[1] Olive Franzese, Congyu Fang, Radhika Garg, Somesh Jha, Nicolas Papernot, Xiao Wang, Adam Dziedzic “Secure Noise Sampling for Differentially Private Collaborative Learning,” Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS’25

[2] Cynthia Dwork, Frank McSherry, Kobbi Nissim, Adam Smith “Calibrating Noise to Sensitivity in Private Data Analysis,” Proceedings of the 3rd Theory of Cryptography Conference, 2006

[3] Frank McSherry, Kunal Talwar “Mechanism Design via Differential Privacy,” 48th Annual IEEE Symposium on Foundations of Computer Science, FOCS’07

[4] Masanobu Kii, Atsunori Ichikawa, Takayuki Miura “Lightweight Two-Party Secure Sampling Protocol for Differential Privacy,” Proceedings on Privacy Enhancing Technologies, PETS’25

[5] Vladimir Kolesnikov, Ranjit Kumaresan, Mike Rosulek, Ni Trieu “Efficient Batched Oblivious PRF with Applications to Privte Set Intersection,” Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS’16

[6] Elette Boyle, Niv Gilboa, Yuval Ishai “Secure Computation with Preprocessing via Function Secret Sharing,” Theory of Cryptography: 17th International Conference, TCC 2019

[7] Elette Boyle, Niv Gilboa, Yuval Ishai “Function Secret Sharing: Improvements and Extensions,” Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS’16

[8] Jack Doerner, Abhi Shelat “Scaling ORAM for Secure Computation,” Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS’17

[9] Arpita Patra, Thomas Schneider, Ajith Suresh, Hossein Yalame “ABY2.0: Improved Mixed-Protocol Secure Two-Party Computation,” 30th USENIX Security Symposium, USENIX Security ’21

[10] osu-crypto “libOTe: A fast and portable C++20 library for Oblivious Transfer extension(OTe),” version 2.2.0, GitHub repository, `https://github.com/osu-crypto/libOTe`, accessed: 2026-06-29

[11] cleverhans-lab “Secure Noise Sampling DP CL,” GitHub repository, `https://github.com/cleverhans-lab/Secure_Noise_Sampling_DP_CL`, accessed: 2026-04-03

付　　録

A.1 PV の証明

各パーティは PV プロトコルの $\delta_p^b, \delta_p^a, \delta_v, -\delta_{pv}$ のシェアに相当する $d_{xor}^{(i)}, d_{add}^{(i)}, m^{(i)}, q^{(i)}$ を準備する．パーティ間で送受信するデータ $e^{(i)} = P^{(i)} \oplus d_{xor}^{(i)}, f^{(i)} = T^{(i)} - m^{(i)}$ から計算できるのは $e^{(0)} \oplus e^{(1)} = P \oplus \delta_p^b, f^{(0)} + f^{(1)} = T - \delta_v$ であり，pv プロトコルの Δ_p^a, Δ_v に相当する．このとき，(6) は次のように展開できる．

$$\begin{aligned} pv &= (\Delta_p^a + (1-2\Delta_p^a)\delta_p^a)(\Delta_v - \delta_v) \\ &= \Delta_p^a(\Delta_v - \delta_v) + (1-2\Delta_p^a)(\delta_p^a\Delta_v - \delta_p^a\delta_v) \\ &= eT^{(i)} + (1-2e)(d_{add}^{(i)} f + q^{(i)}) \end{aligned} \tag{A.1}$$

よって，各パーティの合計は PT になる．

© 2026 Information Processing Society of Japan

$$
\begin{aligned}
\sum_i pv &= eT^{(0)} + (1-2e)(d_{add}^{(0)} f + q^{(0)}) + eT^{(1)} \\
&\quad + (1-2e)(d_{add}^{(1)} f + q^{(1)}) \\
&= eT + (1-2e) f (d_{add} + q) \\
&= eT + (1-2e)(d_{add}(T-m) + d_{add} m) \\
&= (e + (1-2e) d_{add}) T \\
&= PT
\end{aligned}
\tag{A.2}
$$

© 2026 Information Processing Society of Japan